

Simple scheduling

Project 1

von

Thorben Schoenlein

Due day: 2025.11.17

Student ID

32239215

Professor

Seehwan

Tabel of contents

1	Overview	3
2	Task and Requirements	4
2.1	Basic requirement for cpu scheduling	4
3	How the System Works	5
3.1	Initialization	5
3.2	Scheduling	5
3.3	Child Behavior	5
3.4	Logging	5
4	Project Structure	6
5	Advantages	7
6	Disadvantages and Limitations	8
7	Conclusion	9

1 Overview

This project simulates a basic operating system scheduler using the round-robin policy. The simulation demonstrates how a parent (scheduler) process manages multiple child (user) processes, each alternating between CPU and I/O bursts. The system leverages Linux signals, timers, and System V message queues to model time-sliced scheduling and inter-process communication. This project simulates a basic operating system scheduler using the round-robin policy. The simulation demonstrates how a parent (scheduler) process manages multiple child (user) processes, each alternating between CPU and I/O bursts. The system leverages Linux signals, timers, and System V message queues to model time-sliced scheduling and inter-process communication (IPC).

The main objective is to provide a practical, hands-on environment for understanding how an operating system schedules processes, allocates CPU time, and handles process state transitions. The parent process acts as the scheduler, periodically receiving timer interrupts (via `SIGALRM` generated by `setitimer`) and making scheduling decisions based on the round-robin algorithm. Each child process simulates a user workload by looping through randomly generated CPU and I/O bursts, only making progress when granted a time slice by the parent through IPC.

The simulation explicitly maintains a run-queue for processes ready to execute and a wait-queue for processes performing I/O. As processes complete their CPU bursts, they are moved to the wait-queue and, after simulating I/O, are returned to the run-queue with new burst values. All scheduling events, including queue states and process transitions, are logged in a structured, human-readable format to `schedule_dump.txt` using low-level file operations.

This project is designed for educational purposes, making it ideal for students and practitioners who want to bridge the gap between theoretical scheduling concepts and their implementation in real-world systems. The modular code structure also allows for easy extension, enabling experimentation with different scheduling policies, process models, or IPC mechanisms. By working with real UNIX/Linux system calls and signals, users gain valuable experience with the foundational mechanisms that underpin modern operating systems.

2 Task and Requirements

2.1 Basic requirement for cpu scheduling

Parent process:

Create 10 child processes from a parent process. Parent process schedules child processes according to the round-robin scheduling policy. Assume your own scheduling parameters: e.g. time quantum, and timer tick interval. Parent process periodically receives ALARM signal by registering timer event. Students may want to refer to `setitimer` system call. The ALARM signal serves as periodic timer interrupt (or time tick). The parent process maintains run-queue and wait-queue. Run-queue holds child processes that are ready state. Wait-queue holds child processes that are not in ready state. The parent process performs scheduling of its child processes: The parent process accounts for the remaining time quantum of all the child processes. The parent process gives time slice to the child process by sending ipc message through `msgq`. Students may want to refer to `msgget`, `msgsnd`, `msgrcv` system calls. Please note that there is `IPC_NOWAIT` flag. The parent process accounts for the waiting time of all the child processes.

Child process: basic requirement for cpu scheduling

A child process simulates the execution of a user process. Workload consists of infinite loop of dynamic CPU-burst and I/O-burst. The execution begins with two parameters: (`cpu_burst`, `io_burst`). Each value is randomly generated. When a user process receives the time slice from OS, the user process makes progress. To simulate this, the child process makes progress when it is in cpu-burst phase. Besides, the parent process sends ipc message to the currently running child process. When the child process takes ipc message from `msgq`, it decreases cpu-burst value.

The output of the program: <basic requirement for output>

Print out the scheduling operations in the following format: (at time `t`, process `pid` gets cpu time, remaining cpu-burst) run-queue dump, wait-queue dump. Print out all the operations to the following file: `schedule_dump.txt`. Students would like to refer to the following C-library function and syscall: `sprintf`, `open`, `write`, `close`. All the processes should run at least for 1min. Print out the scheduling operations during (0 ~ 10,000) time ticks.

3 How the System Works

3.1 Initialization

The parent forks 10 child processes. Each child enters an infinite loop, waiting for time slices from the parent via the message queue.

3.2 Scheduling

- The parent uses `setitimer` to generate periodic `SIGALRM` signals. On each tick, it:
 - Schedules the next process in the run-queue.
 - Sends a time slice to the selected child.
 - Moves processes to the wait-queue when they request I/O.
 - Moves processes back to the run-queue after I/O completion.
 - Handles IPC messages from children indicating I/O requests.

3.3 Child Behavior

Each child waits for a time slice, simulates CPU work, and requests I/O when its CPU burst is done. After I/O, it generates new random bursts and repeats.

3.4 Logging

Every scheduling event is logged in the required format to `schedule_dump.txt`.

4 Project Structure

The project is organized as follows:

- scheduling-simulation
 - src
 - main.c # Entry point of the scheduling simulation
 - scheduler.c # Implements the round-robin scheduling algorithm
 - scheduler.h # Header file for scheduling functions and data structures
 - process.c # Simulates execution of child processes
 - process.h # Header file for process-related functions and data structures
 - ipc.c # Handles inter-process communication using message queues
 - ipc.h # Header file for IPC-related functions and data structures
 - msg.h # Defines the message structure for IPC communication

5 Advantages

Educational Value

The project provides a clear, hands-on demonstration of core operating system scheduling concepts, including timer interrupts, round-robin scheduling, process state transitions, and inter-process communication. By working with real UNIX/Linux system calls and signals, students gain practical experience that bridges the gap between theory and real-world implementation.

Modular and Extensible Design

The codebase is organized into separate modules for process management, IPC, and scheduling. This modularity makes it easy to understand, maintain, and extend. Developers can experiment with different scheduling algorithms (such as priority scheduling or multi-level feedback queues), add new process behaviors, or introduce additional features like process priorities or starvation prevention.

Realistic Simulation of OS Mechanisms

The use of `setitimer` and `SIGALRM` closely mimics how real operating systems use hardware timer interrupts to drive scheduling decisions. The explicit management of run and wait queues models how an OS kernel tracks process states and transitions.

Clear and Structured Output

All scheduling events and queue states are logged in a human-readable format to `schedule_dump.txt`. This makes it easy to analyze the behavior of the scheduler, debug issues, and visualize how processes move between states over time.

Hands-On IPC Practice

By using System V message queues, the project offers practical experience with IPC mechanisms, which are essential for understanding concurrent and distributed systems.

Foundation for Further Exploration

The simulation serves as a solid starting point for more advanced projects in operating system design, concurrent programming, or distributed systems. It can be adapted for research, teaching, or experimentation with new scheduling policies and process models.

6 Disadvantages and Limitations

No True Parallelism

Although the simulation creates multiple processes, all are ultimately managed by a single CPU core. This means that actual parallel execution is not achieved, and the timing of process execution may not reflect real-world multi-core systems.

Resource Usage

Forking 10 child processes and performing frequent IPC operations can be resource-intensive, especially on systems with limited memory or processing power. This may limit scalability or affect performance on less capable machines.

Simplified Process and I/O Model

The simulation does not account for advanced process features such as priorities, starvation, or aging. I/O bursts are simulated with simple sleep calls, and there is no modeling of real device behavior, blocking, or asynchronous I/O.

No Process Termination or Cleanup

Child processes are not explicitly terminated at the end of the simulation. This can lead to zombie processes or resource leaks if not managed carefully, especially in repeated or long-running experiments.

No Error Recovery or Fault Tolerance

The system assumes reliable IPC and process creation. If a child process crashes or an IPC operation fails, the simulation does not attempt to recover, restart, or handle errors gracefully. This limits its robustness for production or research use.

Fixed Simulation Parameters

The number of processes, time quantum, and burst ranges are hardcoded. Adapting the simulation to different workloads or system sizes requires manual code changes.

Limited Realism

The simulation doesn't model context switch overhead, cache effects, or other hardware-level details that can significantly impact scheduling in real OS.

Output File Size

For long simulations (close to 10,000 ticks), the output file can become quite large, making it harder to analyze or visualize without additional tools.

7 Conclusion

This project delivers a practical and modular simulation of round-robin CPU scheduling using real UNIX/Linux system calls and inter-process communication. By forking 10 child processes and managing them with a parent scheduler, the code demonstrates how a real operating system might allocate CPU time, handle process state transitions, and coordinate process execution using message queues and timer interrupts.

The implementation leverages `setitimer` and `SIGALRM` to simulate periodic timer ticks, which drive the scheduling logic in the parent. Each child process models a user workload by looping through randomly generated CPU and I/O bursts, only making progress when granted a time slice via IPC. The parent maintains explicit run and wait queues, moving processes between them as they request and complete I/O, and logs every scheduling event in a clear, structured format to `schedule_dump.txt` using low-level file operations.

The code is organized for clarity and extensibility, with separate modules for process management, IPC, and scheduling. This makes it easy to adapt or extend the simulation for more advanced scheduling policies, error handling, or process models.

However, the simulation also exposes some inherent limitations of such an approach. All processes are ultimately managed by a single CPU core, so true parallelism is not achieved. The model does not account for priorities, starvation, or complex I/O device behavior, and child processes are not explicitly terminated at the end of the simulation. Additionally, the system assumes reliable IPC and does not attempt to recover from failures.

Despite these limitations, the project provides a strong foundation for understanding the mechanics of CPU scheduling and process management. It bridges the gap between theoretical concepts and practical implementation, making it a valuable educational tool and a starting point for further exploration into operating system design and concurrent programming.